HELLO WORLD

An Introduction to Git for Beginners

BY ANKUSH PATHAK

Open source software has taken the world by storm. It has become the hallmark of an accessible and decentralized internet. Open source software is used with a high degree of trust, even in commercial settings. It is built through a collaboration between people who often live thousands of miles apart. Git is a tool that has made collaborating at a scale convenient. Needless to say, Git itself is open source.

It is a version control system (VCS) or a source code management (SCM) tool. Git and other similar VCSs allow you to record changes to a file(s) in your project and help you avoid files with suffixes like final, final_2, final_final, etc. in your workspace. These files need not necessarily be source code. The aim of any VCS is to allow you to make changes to files fearlessly that is to allow you to go back to a known good state that you saved. Additionally, a VCS enables several people working on a single, shared repository to collaborate with ease.

Linus Torvalds, the creator of the Linux Kernel, needed an SCM tool. He was not fond of the SCM tools available at the time. So, he decided to build his own and thus Git was born [1].

There are several VCSs available today, but in this article, we are going to focus on Git. It has many advantages over other VCSs and is one of the most popular ones out there.

As an aid to walking you through the various commands and uses of Git, I am going to try and build a story of two super-smart developers: Alice and Bob. Alice and Bob are an ambitious bunch and are planning to develop a kernel together (in Python). They have chosen Git as their VCS for this endeavor.

Let's begin.

SETTING UP A CENTRAL GIT SERVER

A Git server allows contributors to publish their code changes on to the server so that others can view and pull them to their local repository. Websites, such as Github and Gitlab, host Git servers that are accessible over the internet. Such servers can be hosted on a workstation in a local network so that peers on that network can access them.

Alice and Bob decide to host a Git server on one of the computers connected to their university network. Both of them would publish their changes to this server so that the other person can pull them and then add their own changes.

Steps to setup a Git server.

1. Create a git user.

```
$ sudo adduser git
#Choose and enter a password. Prompts requesting
other information can be left empty
$ su git #Switch to the git user
```

2. Create a .ssh (note the "." before "ssh") directory. We will use this directory later to grant access to developers.

```
$ cd #This takes you to the git user's home directory
$ mkdir .ssh
$ touch .ssh/authorized _ keys
```

We can set up the permissions of the newly created files to prevent other users from modifying them, but in the interest of brevity, we'll skip that here.

3. Initialize an empty (bare) repository.

```
$ cd
$ mkdir NeoKernel.git
$ cd NeoKernel.git
$ git init --bare
```

4. We now have an empty repository ready for developers to push their updates.

Granting passwordless access to developers.

In order for the developers to be able to push updates to the Git repository without requiring a password, we need to grant them passwordless access rights.

Alice and Bob generate a private-public key pair on their own computer. Their public keys are appended to the "authorized_keys" file on the Git server.

Key pair generation on Alice/Bob's computer:

```
$ ssh-keygen -t ecdsa -b 521
```

The ssh-keygen utility generates a key pair and the public key is saved with the default ".pub" suffix. For more information on key generation visit https://www.ssh.com/ssh/keygen.

Copy the public keys to the Git server.

On the Git server:

```
$ su git
$ cd
$ cat id _ ecdsa _ alice.pub >> ~/.ssh/authorized _ keys
$ cat id _ ecdsa _ bob.pub >> ~/.ssh/authorized _ keys
```

FIRST COMMIT AND PUSH

Alice is eager to start working on their project. So she decides to create a directory skeleton for different modules.

Alice creates a root directory for their project and initializes a Git repository in that directory.

```
#On Alice's computer
$ mkdir NeoKernel && cd NeoKernel
$ git init #Initializes a Git repository
```

She then creates the directory skeleton.

```
$ mkdir MemoryManagement FileSystem
    ProcessScheduling Network
$ ls
FileSystem  MemoryManagement  Network
    ProcessScheduling
```

Let's also add some placeholder files for these modules.

```
$ cat >>MemoryManagement/MemoryManagement.py
print("This is the MemoryManagement module!")

^C
```

Repeat for the rest of the directories.

While Alice has created these directories in a Git repository, Git does not yet track changes to these files. You have to explicitly tell Git to start tracking files. To do that Alice needs to invoke "git add" this tells Git to start tracking all new and modified files in the current directory.

```
$ git status #Before "adding" files
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will
  be committed)

  FileSystem/
  MemoryManagement/
  Network/
  ProcessScheduling/

nothing added to commit but untracked files present
  use "git add" to track)
$ git add .
$ git status #After "adding" files
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

   new file:  FileSystem/FileSystem.py
   new file:  MemoryManagement/MemoryManagement.py
   new file:  Network/Network.py
   new file:  ProcessScheduling/ProcessScheduling.py
```

Git needs to know your name and email before committing these files so that it can properly attribute your commits.

```
$ git config user.email "alice@example.com"
$ git config user.name "Alice"
```

Finally, commit the files.

```
$ git commit -m "First commit:
    Added directory skeleton"
[master (root-commit) 1164d52] First commit:
    Added directory skeleton
 4 files changed, 10 insertions(+)
 create mode 100644 FileSystem/FileSystem.py
 create mode 100644 MemoryManagement/
    MemoryManagement.py
 create mode 100644 Network/Network.py
 create mode 100644 ProcessScheduling/
    ProcessScheduling.py
```

The commit command captures the current state of the repository and adds it to the repository's history. The "-m" flag allows the user to supply a message for the commit. You can use "git checkout" to reinstate your repository to a state recorded under any previous commit.

The commit Alice just made is present only on her local repository copy. For Bob to be able to pull these changes, Alice needs to push them to the Git server that we created.

```
#Replace git _ server _ ip with the Git server's IP
$ git remote add origin git@git _ server _ ip:~/
  NeoKernel.git
#"origin" now refers to the Git server
$ git push origin master
```

The push command tells Git to push the local master branch to the server located at origin. The master branch is a default branch that Git creates for every repository. You can create multiple branches and different branches can have different sets of commits. We will discuss more about branches later.

All that Bob has to do now is pull these changes on to his local computer. Since Alice has already added files to the repository and pushed them, Bob does not need to initialize an empty repository like Alice. But he does not yet have a copy of the repository so he will have to clone the repository locally.

```
#On Bob's computer
#Replace git _ server _ ip with the Git server's IP
$ git clone git@git _ server _ ip:~/NeoKernel.git
$ cd NeoKernel
$ ls
FileSystem  MemoryManagement  Network
    ProcessScheduling
$ git log
commit 1164d52632ff0c850a18a785d896007ba900ce6a
    (HEAD -> master, origin/master, origin/HEAD)
Author: Alice <alice@example.com>
Date:   Sun Feb 23 14:29:33 2020 +0530

   First commit: Added directory skeleton
```

Voila. Just like that, Alice's commit appears on Bob's computer. Bob can now make his own changes or add new files, commit those changes, and push them. Alice will be able to pull them and Bob's changes will appear in Alice's local repository.

Let's quickly look at another push-pull transaction.

```
#On Bob's computer
#Bob adds a function to manage memory
$ cat >> MemoryManagement/MemoryManagement.py
def thisFuncManagesMemoryReallyWell():
  pass

^C

$ git diff #Lines preceded by a single "+"
    are new additions
diff --git a/MemoryManagement/MemoryManagement.py b/
    MemoryManagement/MemoryManagement.py
index 0a0f71a..15647e7 100644
--- a/MemoryManagement/MemoryManagement.py
+++ b/MemoryManagement/MemoryManagement.py
@@ -1,2 +1,5 @@
 print("This is the MemoryManagement module!")

+def thisFuncManagesMemoryReallyWell():
+  pass
+

$ git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what
    will be committed)
  (use "git checkout -- <file>..." to discard
    changes in working directory)

  modified:  MemoryManagement/MemoryManagement.py

no changes added to commit (use "git add" and/or
    "git commit -a")

$ git config user.email "bob@example.com"
$ git config user.name "Bob"
$ git add MemoryManagement/MemoryManagement.py
$ git commit -m "Added memory management function"
$ git push origin master
```

```
#On Alice's computer
$ git pull origin master
$ git log
commit 070ecf36e6d38f5703f3b743db830f7906164184
    (HEAD -> master, origin/master)
Author: Bob <bob@example.com>
Date:   Sun Feb 23 14:56:49 2020 +0530

  Added memory management function

commit 1164d52632ff0c850a18a785d896007ba900ce6a
Author: Alice <alice@example.com>
Date:   Sun Feb 23 14:29:33 2020 +0530

  First commit: Added directory skeleton
```

There are a few things to note here. Whenever a developer modifies files, these changes are "unstaged" and can be viewed by invoking the "git diff" command. After the developer issues the "git add" command the changes become "staged" and then finally they are committed after the developer logs a commit. Only committed changes become a part of the Git history and changes can be committed only after they are staged.

Whenever a Git repository is initialized, Git creates a ".git" directory. Git uses this directory to store all the information it needs to actually execute the commands issued by the user.

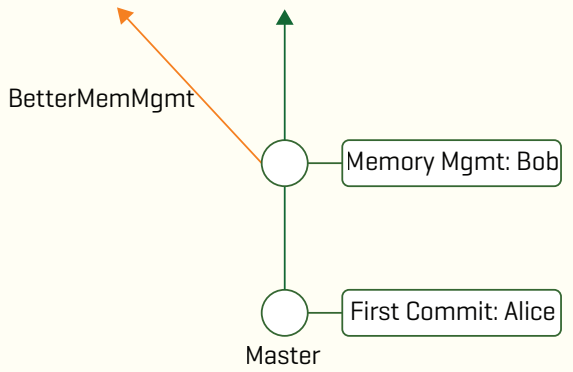
BRANCHES, REBASE, AND MERGE

Alice, after going through Bob's memory management code, thinks she can improve on it. But she does not want to disturb the main (master) branch as Bob plans to move forward with the implementation of a process scheduler on that branch. In such a scenario, Alice can create a new branch and try out her ideas on that branch. This is generally a good idea, because if Alice and Bob are simultaneously making significant changes to code and something breaks, both of them would have to stop their own work until the bad piece of code is identified and rectified.

So Alice creates a new branch from master (see Figure 1).

```
#On Alice's computer
$ git branch #The "*" identifies the current branch
* master
$ git branch BetterMemMgmt #Creates a new branch
$ git checkout BetterMemMgmt #Switch to the new branch
$ git branch
```

Figure 1. The current state of Alice's local repository. Alice and Bob can continue working on the BetterMemMgmt (orange) and master (green) branch respectively without affecting each other's changes committed henceforth.




```
* BetterMemMgmt
  master
```

Alice adds her code to the memory management module and starts testing it.

```
$ git diff
diff --git a/MemoryManagement/MemoryManagement.py b/
    MemoryManagement/MemoryManagement.py
index 15647e7..8faa532 100644
--- a/MemoryManagement/MemoryManagement.py
+++ b/MemoryManagement/MemoryManagement.py
@@ -1,5 +1,7 @@
 print("This is the MemoryManagement module!")

 def thisFuncManagesMemoryReallyWell():
+   #Alice's code here, that she thinks is
+   #better than the existing approach
    pass

$ git add MemoryManagement/MemoryManagement.py
$ git commit -m "Better Memory Management
    (experimental)"
#Push the BetterMemMgmt branch to origin
$ git push origin BetterMemMgmt
```

Meanwhile, Bob is working on the process scheduler.

```
#On Bob's computer
$ cat >> ProcessScheduling/ProcessScheduling.py
def worldsBestProcManager():
   pass
^C
$ git diff
diff --git a/ProcessScheduling/ProcessScheduling.py
    b/ProcessScheduling/ProcessScheduling.py
index 10937e6..f2cc71b 100644
--- a/ProcessScheduling/ProcessScheduling.py
+++ b/ProcessScheduling/ProcessScheduling.py
@@ -1,3 +1,5 @@
 print("This is the ProcessScheduling module!")
+def worldsBestProcManager():
+   pass

$ git add ProcessScheduling/ProcessScheduling.py
$ git commit -m "Implemented ProcessScheduler"
$ git push origin master
```

Bob can pull Alice's branch from origin to his local repository, but that won't affect the master branch.

```
$ git fetch origin
$ git checkout BetterMemMgmt
$ git log
commit c7147ce6fef1bfda0190427bcdc511125e87626d
    (HEAD -> BetterMemMgmt, origin/BetterMemMgmt)
Author: Alice <alice@example.com>
Date:   Sun Feb 23 16:57:16 2020 +0530

   Better Memory Management (experimental)

commit 070ecf36e6d38f5703f3b743db830f7906164184
Author: Bob <bob@example.com>
Date:   Sun Feb 23 14:56:49 2020 +0530

   Added memory management function

commit 1164d52632ff0c850a18a785d896007ba900ce6a
Author: Alice <alice@example.com>
Date:   Sun Feb 23 14:29:33 2020 +0530

   First commit: Added directory skeleton

#Master branch is untouched
$ git checkout master
$ git log
commit 0c9f7786d1456eb15bbae774b35fadca34b40a5e
    (HEAD -> master, origin/master, origin/HEAD)
Author: Bob <bob@example.com>
Date:   Sun Feb 23 17:05:38 2020 +0530

   Implemented ProcessScheduler

commit 070ecf36e6d38f5703f3b743db830f7906164184
Author: Bob <bob@example.com>
Date:   Sun Feb 23 14:56:49 2020 +0530

   Added memory management function

commit 1164d52632ff0c850a18a785d896007ba900ce6a
Author: Alice <alice@example.com>
Date:   Sun Feb 23 14:29:33 2020 +0530

   First commit: Added directory skeleton
```

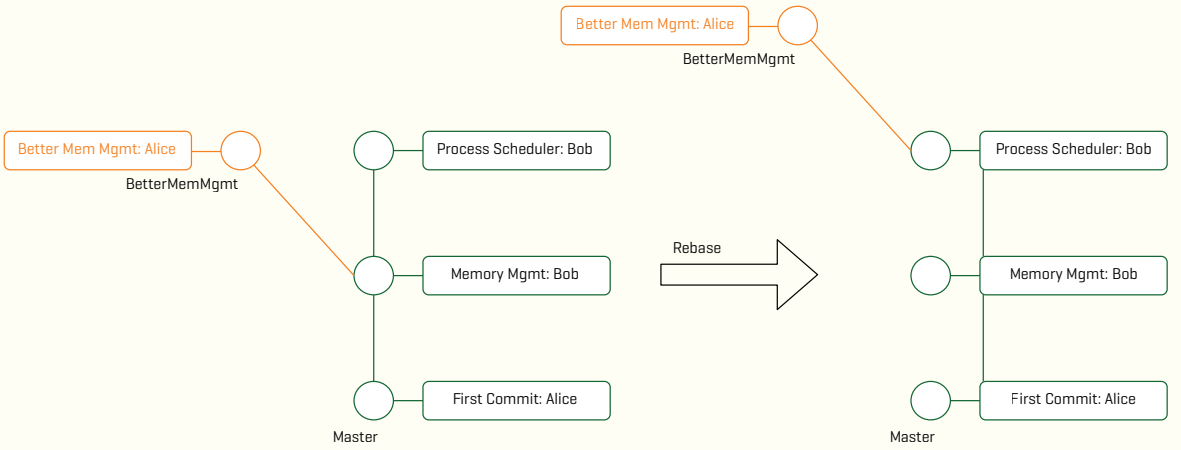
Alice has tested her changes thoroughly. She realizes Bob has just implemented a process scheduler, and she is not sure if her new memory manager is compatible with it. So she wants to try out her new memory management algorithm with Bob's newly implemented process scheduler. Essentially, Alice wants to include the latest commits in the master branch in the BetterMemMgmt branch, she achieves that using rebase.

```
#On Alice's computer
#First, update the master branch
$ git checkout master
$ git pull origin master
$ git checkout BetterMemMgmt
$ git rebase master
```

Before we talk about rebase let's discuss the difference between "pull" and "fetch." The command "git fetch origin" will download all the updates from origin to your local repository, including new branches and updates to existing local branches. It will only download them but won't actually integrate them in to local branches. On the other hand, "git pull origin master" will download the master branch from the remote server and apply the downloaded updates to the current local branch, which in Alice's case was the local master branch.

The rebase command updates the current branch to include all of the master branch's commits and then replays the current branch's changes on top (see Figure 2).

Figure 2. Rebase. Changes (commits) made by Alice on top of Bob's "Memory Mgmt" commit are applied on top of Bob's latest "Process Scheduler" commit.




```
#Git log will now include Bob's process scheduler commit,
and Alice's memory management commit on top of it
$ git log
commit cf6ad83fdce83617c229c019941a09a2837a8082
    (HEAD -> BetterMemMgmt)
Author: Alice <alice@example.com>
Date:   Sun Feb 23 16:57:16 2020 +0530

   Better Memory Management (experimental)

commit 0c9f7786d1456eb15bbae774b35fadca34b40a5e
    (origin/master, master)
Author: Bob <bob@example.com>
Date:   Sun Feb 23 17:05:38 2020 +0530

   Implemented ProcessScheduler

commit 070ecf36e6d38f5703f3b743db830f7906164184
Author: Bob <bob@example.com>
Date:   Sun Feb 23 14:56:49 2020 +0530

   Added memory management function

commit 1164d52632ff0c850a18a785d896007ba900ce6a
Author: Alice <alice@example.com>
Date:   Sun Feb 23 14:29:33 2020 +0530

   First commit: Added directory skeleton
```

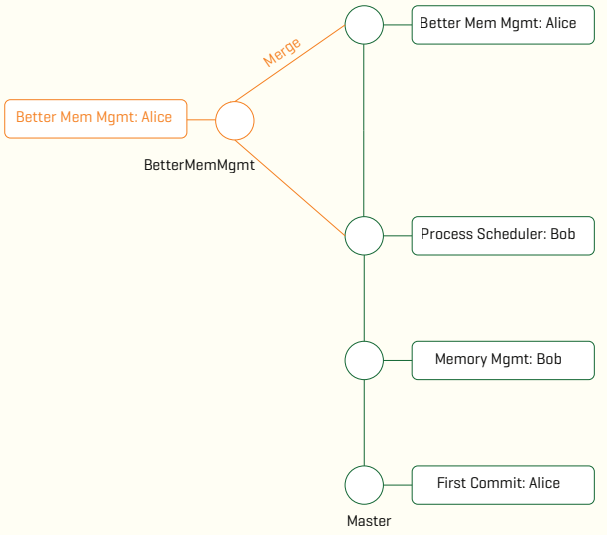
Alice's further testing was satisfactory and her new approach of memory management was compatible with Bob's process scheduler. She decided it was time to include her changes in the master branch.

```
#On Alice's computer
$ git checkout master
$ git merge BetterMemMgmt
$ git push origin master
```

All of Alice's changes are now also on the master branch locally as well on the Git server (see Figure 3). If you view the master and BetterMemMgmt log, both are identical. BetterMemMgmt's utility has ended here and it can be deleted.

Let's leave Alice and Bob to their devices now.

Figure 3. Merge in master. Changes under Alice's "Better Mem Mgmt" commit are available on the master branch now.



If you realize, all the Git commands that we discussed except for push, pull, fetch, and clone work without requiring any connection to the server. We explored Git using its command line interface (CLI), but there are many graphical interfaces available for people not comfortable with a CLI.

This article has illustrated how two people can collaborate using Git. But Git is able to scale seamlessly and provide its services to large projects with even larger teams. Git has become an essential component of contemporary software

development processes. Often, continuous integration/continuous deployment (CI/CD) pipelines are configured such that a commit on a specific Git branch triggers a build followed by an automatic test suite. If the test suite runs successfully then the build is automatically deployed into production.

This article has barely scratched the surface of Git. I did not cover many commands or even different ways the commands that we discussed can be used. If all this was too heavy to digest, don't fret, even Andrew Morton, who was one of the lead developers on the Linux Kernel project, once said that Git is "expressly designed to make you feel less intelligent than you thought you were"[2].

I would like to leave you with an excerpt from Git's man page.

```
NAME
    git - the stupid content
    tracker
```

Software Specification

I have tested all the discussed commands on Ubuntu 18.04 with Git version 2.17.1

Further Reading

Pro Git (https://git-scm.com/book/en/v2) is a fantastic book to learn Git. It comprehensively covers all aspects of Git a developer would ever need.

References

[1] 10 Years of Git: An Interview with Git Creator Linus Torvalds. *The Linux Foundation*. (April 6, 2015); https://www.linuxfoundation.org/blog/2015/04/10-years-of-git-an-interview-with-git-creator-linus-torvalds

[2] The Greatness of Git. *The Linux Foundation*. (February 28, 2012); https://www.linuxfoundation.org/blog/2012/02/the-greatness-of-git

Biography

Ankush is currently working with Persistent Systems as a project engineer. He has been an *XRDS* volunteer for more than three years. He someday hopes to contribute to projects that can empower the underprivileged people of the world through technology.

DOI: 10.1145/3398459

Copyright held by author.

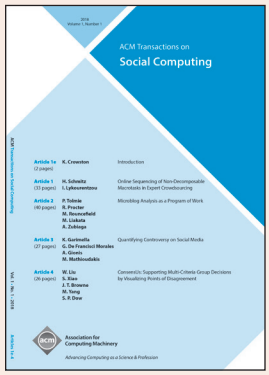
Association for Computing Machinery

ACM Transactions on
Social Computing (TSC)

Open for Submissions

Seeks to publish work that covers the full spectrum of social computing including theoretical, empirical, systems, and design research contributions.

ACM Transactions on Social Computing (TSC) seeks to publish work that covers the full spectrum of social computing including theoretical, empirical, systems, and design research contributions. The editorial perspective is that social computing is fundamentally about computing systems and techniques in which users interact, directly or indirectly, with what they believe to be other users or other users' contributions. TSC welcomes research employing a wide range of methods to advance the tools, techniques, understanding, and practice of social computing, including: theoretical, algorithmic, empirical, experimental, qualitative, quantitative, ethnographic, design, and engineering research. Social computing will continue to be shaped by foundational algorithmic, econometric, psychological, sociological, and social science research and these broad-based perspectives will continue to have a profound influence on how social computing systems are designed, built and how they grow.



For further information and to submit your manuscript, visit tsc.acm.org